
机器视觉技术 练习题合集

含答案版

模拟练习卷 · 深度学习专项 · Faster R-CNN 专项

目录

第一编 机器视觉技术模拟练习卷 (一)	1
1.1 一、选择题 (10 题, 每题 2 分)	1
1.2 二、判断题 (5 题, 每题 2 分)	2
1.3 三、简答题 (5 题, 每题 6 分)	3
1.4 四、解答题 (2 题, 每题 10 分)	3
1.5 五、程序题 (2 题, 每题 10 分)	5
1.6 参考答案	7
1.7 一、选择题答案	7
1.8 二、判断题答案	7
1.9 三、简答题参考要点	7
1.10 四、解答题答案	9
1.11 五、程序题参考答案	10
第二编 机器视觉技术模拟练习卷 (二)	14
2.1 一、选择题 (10 题, 每题 2 分)	14
2.2 二、判断题 (5 题, 每题 2 分)	16
2.3 三、简答题 (5 题, 每题 6 分)	16
2.4 四、解答题 (2 题, 每题 10 分)	17
2.5 五、程序题 (2 题, 每题 10 分)	19
2.6 参考答案	21
2.7 一、选择题答案	21
2.8 二、判断题答案	21
2.9 三、简答题参考答案	22
2.10 四、解答题参考答案	24
2.11 五、程序题参考答案	27
第三编 深度学习专项题库	32
3.1 一、问答题专项	32
3.2 二、解答题专项	39
3.3 三、综合性问答题	48

第四编 Faster R-CNN 专项题库	50
4.1 一、Anchor 专项问答题	50
4.2 二、ROI Pooling 专项问答题	54
4.3 三、综合问答题	57
4.4 四、计算与解答题	59
使用说明	62

第一编 机器视觉技术模拟练习卷（一）

本卷满分 100 分。建议先完成练习，再对照答案复盘知识点与计算过程。

1.1 一、选择题（10 题，每题 2 分）

1. 一幅大小为 $M \times N$ 的灰度图像共有 L 个灰度级，其直方图 $h(k)$ 表示
 - A. 灰度值为 k 的像素位置
 - B. 灰度值为 k 的像素数量
 - C. 图像中所有像素的平均灰度
 - D. 相邻像素之间的灰度差
2. 以下关于图像采样和量化的说法，正确的是
 - A. 采样决定灰度级数量，量化决定空间分辨率
 - B. 采样和量化都只影响空间分辨率
 - C. 采样决定空间分辨率，量化决定灰度分辨率
 - D. 量化级数越少，图像灰度变化越平滑
3. 对椒盐噪声具有较好抑制作用，并且能够较好保留边缘的滤波器是
 - A. 均值滤波器
 - B. 高斯滤波器
 - C. 中值滤波器
 - D. 拉普拉斯滤波器
4. Canny 边缘检测中，非极大值抑制的主要作用是
 - A. 增强图像对比度
 - B. 将较宽的边缘细化为单像素边缘
 - C. 消除所有弱边缘
 - D. 对图像进行高斯平滑
5. 对二值图像先进行腐蚀，再进行膨胀，得到的形态学运算是
 - A. 闭运算
 - B. 开运算
 - C. 顶帽运算
 - D. 边界提取

6. SIFT 特征具有尺度不变性的主要原因是

- A. 使用了灰度直方图
- B. 在高斯尺度空间中检测极值点
- C. 使用了非极大值抑制
- D. 对描述子进行了二值化

7. HOG 特征在方向直方图中累加的通常是

- A. 像素数量
- B. 像素灰度值
- C. 梯度幅值
- D. 像素坐标

8. 在 Hough 直线检测中, 图像空间中的一个边缘点, 在参数空间中对应

- A. 一个点
- B. 一条曲线
- C. 一个矩形区域
- D. 一条固定直线

9. 输入特征图大小为 $32 \times 32 \times 3$, 使用 16 个 3×3 卷积核, 步长为 1, 填充为 1, 则输出特征图大小为

- A. $30 \times 30 \times 16$
- B. $32 \times 32 \times 16$
- C. $32 \times 32 \times 3$
- D. $34 \times 34 \times 16$

10. Faster R-CNN 中, ROI Pooling 的主要作用是

- A. 生成候选框
- B. 将不同大小的候选区域转换为固定大小的特征
- C. 计算 Anchor 与真实框的 IoU
- D. 删除重叠候选框

1.2 二、判断题 (5 题, 每题 2 分)

1. 均值滤波属于非线性滤波, 中值滤波属于线性滤波。

()

2. Canny 边缘检测中的双阈值操作, 可以根据强边缘连接情况保留部分弱边缘。

()

3. 图像经过旋转后, SIFT 关键点的主方向也会相应改变, 因此其描述子仍能保持较好的旋转不变性。

()

4. HOG 特征通常直接对整幅图像的梯度直方图进行一次归一化, 不需要划分 Cell 和 Block。

()

5. ResNet 中的残差连接可以缓解深层网络的退化问题, 并有利于梯度传播。

()

1.3 三、简答题 (5 题, 每题 6 分)

1. 简述线性滤波和非线性滤波的区别, 并分别举出两个典型滤波器。

2. 简述 Canny 边缘检测的主要步骤。

要求写出以下内容:

- 图像平滑;
- 梯度计算;
- 非极大值抑制;
- 双阈值检测;
- 边缘连接。

3. 简述 SIFT 特征检测具有尺度不变性和旋转不变性的原因。

4. 简述 HOG 特征的生成步骤, 并说明为什么方向直方图中累加的是梯度幅值, 而不是像素数量。

5. 简述 ResNet 残差结构的基本思想, 并说明恒等映射为什么有利于深层网络训练。

1.4 四、解答题 (2 题, 每题 10 分)

1. 卷积运算与边缘检测

给定图像区域

$$I = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 5 & 5 \\ 1 & 5 & 9 \end{bmatrix}$$

以及水平方向 Sobel 算子

$$G_x = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

垂直方向 Sobel 算子

$$G_y = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

回答:

- 计算该区域中心位置的 g_x 和 g_y ;
计算梯度幅值

$$M = \sqrt{g_x^2 + g_y^2}$$

计算梯度方向

$$\theta = \text{atan2}(g_y, g_x)$$

说明该位置更可能属于水平方向边缘还是垂直方向边缘。

2. 卷积神经网络尺寸与参数计算

某卷积神经网络的一部分如下:

输入: $32 \times 32 \times 3$

Conv1:

卷积核大小 3×3

输出通道数 16

$$\text{stride} = 1 \text{ padding} = 1$$

MaxPool:

窗口大小 2×2

$$\text{stride} = 2$$

Conv2:

卷积核大小 3×3

输出通道数 32

$$\text{stride} = 1 \text{ padding} = 0$$

回答:

- Conv1 输出特征图尺寸;

- Conv1 可学习参数数量，包含偏置；
- MaxPool 输出特征图尺寸；
- Conv2 输出特征图尺寸；
Conv2 可学习参数数量，包含偏置。

卷积输出尺寸公式为：

$$H_{out} = \text{floor}((H_{in} + 2P - K)/S) + 1$$

1.5 五、程序题 (2 题，每题 10 分)

1. OpenCV 非线性滤波程序题

使用 C++ 和 OpenCV 编写函数，对输入灰度图像进行 3×3 中值滤波。

要求：

- 不允许直接调用 `cv::medianBlur`；
- 对每个非边界像素，取其 3×3 邻域中的 9 个灰度值；
- 将 9 个灰度值排序；
- 使用中间值作为输出；

边界像素直接复制原图像值。

函数形式：

```
cv::Mat myMedianFilter(const cv::Mat& src);
```

2. PyTorch 残差模块程序题

使用 PyTorch 编写一个 ResNet 基本残差模块 BasicBlock。

要求：

- 包含两个 3×3 卷积层；
- 每个卷积层后使用 BatchNorm；
- 第一个卷积层后使用 ReLU；
- 将卷积结果与输入恒等映射相加；
- 相加之后再次使用 ReLU；

当输入通道和输出通道不同时，使用 1×1 卷积调整维度。

函数框架：

```
class BasicBlock(nn.Module):  
    def __init__(self, in_channels, out_channels, stride=1):
```

```
pass

def forward(self, x):
    pass
```

1.6 参考答案

1.7 一、选择题答案

题号 答案 核心知识点

- 1 B 直方图统计灰度频数
- 2 C 采样决定空间，量化决定灰度
- 3 C 中值滤波抑制椒盐噪声
- 4 B 非极大值抑制细化边缘
- 5 B 先腐蚀后膨胀为开运算
- 6 B 高斯尺度空间
- 7 C 梯度幅值加权投票
- 8 B 一个图像点对应参数空间中的曲线
- 9 B 填充后尺寸不变
- 10 B 不同 ROI 转为固定尺寸

1.8 二、判断题答案

错误。均值滤波是线性滤波，中值滤波是非线性滤波。

正确。与强边缘连通的弱边缘可以被保留。

正确。SIFT 以关键点主方向作为描述子的参考方向。

错误。HOG 要划分 Cell 和 Block，并进行局部归一化。

正确。

1.9 三、简答题参考要点

1. 线性滤波与非线性滤波

线性滤波的输出是邻域像素的线性加权和，可以用卷积表示，例如：

- 均值滤波；
- 高斯滤波。

非线性滤波的输出不能写成邻域像素的固定线性加权和，例如：

- 中值滤波；
- 最大值滤波；
- 最小值滤波。

中值滤波对椒盐噪声效果好，并且比均值滤波更容易保留边缘。

2. Canny 边缘检测步骤

- 使用高斯滤波消除噪声;
- 计算图像在水平和垂直方向上的梯度;
- 计算梯度幅值和梯度方向;
- 沿梯度方向进行非极大值抑制, 使边缘变细;
- 使用高、低双阈值区分强边缘、弱边缘和非边缘;
根据连通性保留与强边缘相连的弱边缘。

3. SIFT 的不变性

**** 尺度不变性: ****SIFT 建立高斯尺度空间, 并在不同尺度的 DoG 图像中检测极值点。同一目标缩放后, 仍可在相应尺度找到相同关键点。

**** 旋转不变性: ****SIFT 根据关键点邻域梯度方向直方图确定主方向, 并以主方向作为描述子参考方向。因此图像旋转时, 参考方向也随之旋转。

4. HOG 生成步骤

- 对输入图像进行归一化或 Gamma 校正;
- 计算每个像素的梯度幅值和梯度方向;
- 将图像划分为多个 Cell;
- 在每个 Cell 中建立梯度方向直方图;
- 将若干相邻 Cell 组成 Block;
- 对 Block 中的特征进行归一化;

连接所有 Block 特征形成 HOG 描述子。

方向直方图中使用梯度幅值作为投票权重, 是因为梯度越大的像素通常对应越明显的边缘, 应当对特征贡献更大。

5. ResNet 基本思想

普通网络直接学习目标映射 $H(x)$, ResNet 改为学习残差:

$$F(x)=H(x)-x$$

于是网络输出为:

$$y=F(x)+x$$

当增加的网络层没有学习到有效特征时, 只需要使 $F(x)\approx 0$, 整个模块就接近恒等映射, 不会明显破坏原有特征。

跳跃连接还为梯度提供了更直接的传播路径, 可以缓解梯度消失和网络退化问题。

1.10 四、解答题答案

1. Sobel 计算

计算 g_x

$$g_x = (-1) \times 1 + 0 \times 1 + 1 \times 1 + (-2) \times 1 + 0 \times 5 + 2 \times 5 + (-1) \times 1 + 0 \times 5 + 1 \times 9 = 16$$

计算 g_y

$$g_y = (-1) \times 1 + (-2) \times 1 + (-1) \times 1 + 0 + 1 \times 1 + 2 \times 5 + 1 \times 9 = 16$$

梯度幅值

$$M = \sqrt{16^2 + 16^2} = 16 \sqrt{2} \approx 22.63$$

梯度方向

$$\theta = \text{atan2}(g_y, g_x)$$

梯度方向为 45

。

，说明该位置亮度变化同时包含明显的水平变化和垂直变化，属于斜向边缘，而不是纯水平或纯垂直边缘。

注意：梯度方向与边缘延伸方向垂直。

2. CNN 计算

Conv1 输出尺寸

$$H_{\text{out}} = (32 + 2 \times 1 - 3) / 1 + 1 = 32$$

因此：

$$32 \times 32 \times 16$$

Conv1 参数量

每个卷积核参数：

$$3 \times 3 \times 3$$

共有 **16** 个卷积核，并且每个卷积核有一个偏置：

$$(3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = 448$$

池化后尺寸

$$16 \times 16 \times 16$$

Conv2 输出尺寸

$$H_{\text{out}} = (16 - 3) / 1 + 1 = 14$$

因此:

$14 \times 14 \times 32$

Conv2 参数量

$$(3 \times 3 \times 16 + 1) \times 32 = 145 \times 32 = 4640$$

1.11 五、程序题参考答案

1. 手写中值滤波

```
#include <opencv2/opencv.hpp>
#include <algorithm>
#include <stdexcept>
#include <vector>

cv::Mat myMedianFilter(const cv::Mat& src)
{
    if (src.empty()) {
        throw std::invalid_argument(" 输入图像不能为空");
    }

    if (src.type() != CV_8UC1) {
        throw std::invalid_argument(" 输入图像必须是 8 位单通道灰度图像");
    }

    cv::Mat dst = src.clone();

    for (int i = 1; i < src.rows - 1; ++i) {
        for (int j = 1; j < src.cols - 1; ++j) {
            std::vector<uchar> values;
            values.reserve(9);

            for (int di = -1; di <= 1; ++di) {
                for (int dj = -1; dj <= 1; ++dj) {
                    values.push_back(
                        src.at<uchar>(i + di, j + dj)
                    );
                }
            }

            std::sort(values.begin(), values.end());

            // 排序后第 5 个元素, 即下标 4
            dst.at<uchar>(i, j) = values[4];
        }
    }
}
```

```
    }

    return dst;
}
```

```
int main()
{
```

```
    cv::Mat image =
        cv::imread("input.png", cv::IMREAD_GRAYSCALE);

    if (image.empty()) {
        return -1;
    }

    cv::Mat result = myMedianFilter(image);

    cv::imshow("Original", image);
    cv::imshow("Median Filter", result);
    cv::waitKey(0);

    return 0;
}
```

核心语句是：

```
std::sort(values.begin(), values.end());
dst.at<uchar>(i, j) = values[4];
```

因为 9 个数字排序后，中值是下标为 4 的元素。

2. ResNet 基本残差模块

```
import torch
import torch.nn as nn

class BasicBlock(nn.Module):
    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        stride: int = 1
    ):
        super().__init__()
```

```
self.conv1 = nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size=3,
    stride=stride,
    padding=1,
    bias=False
)

self.bn1 = nn.BatchNorm2d(out_channels)
self.relu = nn.ReLU(inplace=True)

self.conv2 = nn.Conv2d(
    out_channels,
    out_channels,
    kernel_size=3,
    stride=1,
    padding=1,
    bias=False
)

self.bn2 = nn.BatchNorm2d(out_channels)

# 输入、输出尺寸不一致时调整恒等分支
if stride != 1 or in_channels != out_channels:
    self.shortcut = nn.Sequential(
        nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=1,
            stride=stride,
            bias=False
        ),
        nn.BatchNorm2d(out_channels)
    )
else:
    self.shortcut = nn.Identity()

def forward(self, x: torch.Tensor) -> torch.Tensor:
    identity = self.shortcut(x)

    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.conv2(out)
    out = self.bn2(out)
```

```
        out = out + identity
        out = self.relu(out)

    return out

if __name__ == "__main__":
    x = torch.randn(4, 64, 32, 32)

    block = BasicBlock(
        in_channels=64,
        out_channels=128,
        stride=2
    )

    y = block(x)

    print(" 输入尺寸: ", x.shape)
    print(" 输出尺寸: ", y.shape)
```

输出尺寸为:

输入尺寸: torch.Size([4, 64, 32, 32])

输出尺寸: torch.Size([4, 128, 16, 16])

其中 **1×1** 卷积同时完成:

- 通道数由 64 变为 128;
 - 空间尺寸由 32×32 变为 16×16;
- 保证恒等分支能够与主分支相加。

第二编 机器视觉技术模拟练习卷（二）

本卷满分 100 分。建议先完成练习，再对照答案复盘知识点与计算过程。

2.1 一、选择题（10 题，每题 2 分）

1. 一幅 8 位灰度图像最多可以表示的灰度级数量为
 - A. 8
 - B. 16
 - C. 255
 - D. 256
2. 关于图像直方图，下列说法错误的是
 - A. 直方图能够反映图像中各灰度级的像素数量
 - B. 直方图可以反映图像整体的明暗分布
 - C. 直方图完整保留了像素的空间位置信息
 - D. 两幅不同的图像可能具有相同的直方图
3. 与均值滤波相比，高斯滤波的主要特点是
 - A. 所有邻域像素权重完全相同
 - B. 距离中心越近的像素通常权重越大
 - C. 属于非线性滤波
 - D. 主要用于检测直线
4. 拉普拉斯算子进行边缘检测时，对噪声比较敏感，主要原因是
 - A. 拉普拉斯算子只计算图像平均值
 - B. 拉普拉斯算子属于二阶微分算子
 - C. 拉普拉斯算子不能用于灰度图像
 - D. 拉普拉斯算子不包含卷积运算
5. 对二值图像先进行膨胀，再进行腐蚀，称为
 - A. 开运算
 - B. 闭运算
 - C. 边界提取
 - D. 骨架提取

6. SIFT 算法在检测尺度空间极值点后，需要删除低对比度关键点，主要是为了

- A. 减少不稳定关键点
- B. 增大图像尺寸
- C. 保证所有关键点位于边缘
- D. 将彩色图像转换为灰度图像

7. HOG 特征中，对相邻若干 Cell 组成的 Block 进行归一化，主要目的是

- A. 增加图像的空间尺寸
- B. 减小光照和局部对比度变化的影响
- C. 将梯度方向变成灰度值
- D. 删除所有弱梯度

8. Hough 直线变换通常采用的直线参数方程是

- A. $y=kx+b$
- B. $x^2+y^2=r^2$
- C. $\rho=x\cos\theta+y\sin\theta$
- D. $y=ax^2+bx+c$

9. 输入特征图大小为 $28\times 28\times 3$ ，使用 6 个 5×5 卷积核，步长为 1，不填充。输出特征图尺寸和可学习参数量分别为

- A. $24\times 24\times 6$, 456
- B. $28\times 28\times 6$, 450
- C. $24\times 24\times 3$, 456
- D. $32\times 32\times 6$, 450

假设每个卷积核包含一个偏置参数。

10. U-Net 中编码器与解码器之间使用跳跃连接的主要作用是

- A. 删除浅层特征
- B. 融合浅层定位信息和深层语义信息
- C. 生成目标检测 Anchor
- D. 将所有特征图压缩为一维向量

2.2 二、判断题 (5 题, 每题 2 分)

1. 图像的曼哈顿距离 D

4

一定不小于棋盘距离 D

8

。

()

2. 高斯滤波可以在完全不损失任何边缘信息的情况下去除所有噪声。

()

3. 对同一二值图像进行膨胀时, 结构元素越大, 前景区域通常扩张得越明显。

()

4. SIFT 算法中的 DoG 图像可以由相邻尺度的高斯平滑图像相减得到。

()

5. Faster R-CNN 中的 RPN 需要判断 Anchor 是否包含目标, 并对候选框位置进行回归。

()

2.3 三、简答题 (5 题, 每题 6 分)

1. 简述图像采样和量化的含义, 并分别说明采样不足和量化级数不足可能造成的现象。

2. 为什么在进行边缘检测之前, 通常需要先对图像进行平滑处理?

要求结合微分运算和图像噪声进行说明。

3. 简述二值图像开运算和闭运算的执行顺序及主要作用。

4. 简述 HOG 特征的基本生成过程, 并说明 Cell 和 Block 分别起什么作用。

5. 简述 Faster R-CNN 的基本检测流程, 并说明 Anchor、RPN 和 ROI Pooling 分别承担什么作用。

2.4 四、解答题 (2 题, 每题 10 分)

1. 二值形态学与 Hough 变换

(1) 二值形态学计算

给定二值图像

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

结构元素为

$$B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

结构元素的原点位于中心, 图像外部像素视为 0。

计算:

- 图像 I 被 B 膨胀后的结果;
图像 I 被 B 腐蚀后的结果。

(2) Hough 直线检测

图像中存在以下 6 个边缘点:

(1,1),(2,2),(3,3),(2,0),(2,1),(2,3)

采用 Hough 直线方程

$$\rho = x \cos \theta + y \sin \theta$$

只考虑以下两个角度:

$$\theta = 0^\circ, \theta = 135^\circ$$

回答:

当 $\theta = 0$

。

时, 哪个 ρ 值获得的票数最多?

当 $\theta = 135$

。

时, 哪个 ρ 值获得的票数最多?

写出检测到的两条直线。

2. CNN 特征尺寸与参数计算

某卷积神经网络结构如下：

输入图像： $64 \times 64 \times 3$

Conv1:

卷积核大小： 5×5

输出通道数：8

stride = 2 padding = 2

MaxPool:

窗口大小： 2×2

stride = 2

Conv2:

卷积核大小： 3×3

输出通道数：16

stride = 1 padding = 1

Flatten

全连接层：

输出神经元数量为 10

回答：

- Conv1 输出特征图尺寸；
- Conv1 参数量，包含偏置；
- MaxPool 输出特征图尺寸；
- Conv2 输出特征图尺寸；
- Conv2 参数量，包含偏置；
- 全连接层参数量；

整个网络中上述可学习参数总数。

卷积输出尺寸公式为：

$$H_{out} = \text{floor}((H_{in} + 2P - K)/S) + 1$$

2.5 五、程序题 (2 题, 每题 10 分)

1. OpenCV 最大值滤波

使用 C++ 和 OpenCV 编写一个 3×3 最大值滤波函数。

最大值滤波的定义为：对于每个像素，取其 3×3 邻域内的最大灰度值作为输出。

要求：

- 不允许调用 `cv::dilate`;
 - 不允许调用其他现成最大值滤波函数;
 - 输入为 8 位单通道灰度图像;
 - 对非边界像素遍历 3×3 邻域;
- 边界像素直接复制原图像值。

函数形式如下：

```
cv::Mat maxFilter3x3(const cv::Mat& src);
```

2. PyTorch 简化 U-Net

使用 PyTorch 编写一个简化的 U-Net 网络。

网络结构如下：

输入图像

↓

DoubleConv: $3 \rightarrow 64$

↓

最大池化

↓

DoubleConv: $64 \rightarrow 128$

↓

转置卷积上采样: $128 \rightarrow 64$

↓

与编码器对应特征拼接

↓

DoubleConv: $128 \rightarrow 64$

↓

1×1 卷积输出分类结果

要求:

- DoubleConv 包含两个 3×3 卷积;
- 每个卷积后使用 BatchNorm 和 ReLU;
- 使用最大池化完成下采样;
- 使用转置卷积完成上采样;
- 使用 torch.cat 实现跳跃连接;

最后一层使用 1×1 卷积输出 num_classes 个通道。

2.6 参考答案

2.7 一、选择题答案

题号 答案 知识点

1 D 8 位图像有 2

8

=256 个灰度级

2 C 直方图不包含空间位置信息

3 B 高斯核中心权重大

4 B 二阶微分对噪声敏感

5 B 先膨胀后腐蚀是闭运算

6 A 删除低对比度不稳定点

7 B 减小光照和对比度影响

8 C Hough 直线极坐标形式

9 A 尺寸与参数量计算

10 B 融合浅层与深层特征

第 9 题计算

输出尺寸:

$$H_{out} = \text{floor}((H_{in} + 2P - K)/S) + 1$$

所以输出为:

$$24 \times 24 \times 6$$

参数量:

$$(5 \times 5 \times 3 + 1) \times 6 = 76 \times 6 = 456$$

2.8 二、判断题答案

正确

对于两个像素间坐标差 $(\Delta x, \Delta y)$:

D_4

$$= |\Delta x| + |\Delta y|$$

D_8

$$=\max(|\Delta x|, |\Delta y|)$$

因此：

$$D_4 \geq D_8$$

错误。平滑噪声的同时通常也会使部分边缘变模糊。

正确。较大的结构元素通常会使前景扩张范围更大。

正确。DoG 是相邻高斯尺度图像之差。

正确。RPN 包含前景与背景分类以及边界框回归。

2.9 三、简答题参考答案

1. 采样和量化

采样是将连续图像的空间坐标离散化，决定图像的空间分辨率。

采样点过少可能出现：

- 图像细节丢失；
- 锯齿；
- 混叠现象。

量化是将连续的亮度或颜色值离散为有限个等级，决定灰度分辨率。

量化级数过少可能出现：

- 灰度层次减少；
- 平滑区域出现假轮廓；
- 图像色阶不连续。

2. 边缘检测前进行平滑的原因

边缘检测通常需要进行一阶或二阶微分，而微分运算会放大图像中的高频成分。

图像噪声通常也属于高频成分，因此直接进行微分可能产生大量虚假边缘。

先采用高斯滤波等方法进行平滑，可以：

- 抑制噪声；
- 减少虚假梯度；
- 提高边缘检测的稳定性。

但是平滑过强也会使真实边缘变得模糊，因此需要在去噪和保留边缘之间进行权衡。

3. 开运算和闭运算

开运算

先腐蚀，后膨胀：

$$A \circ B = (A \ominus B) \oplus B$$

主要作用：

- 删除较小的前景噪声；
- 断开细小连接；
平滑物体外边界。

闭运算

先膨胀，后腐蚀：

$$A \bullet B = (A \oplus B) \ominus B$$

主要作用：

- 填补较小的内部孔洞；
- 连接相邻目标；
弥合狭窄断裂。

4. HOG 特征生成过程

- 对图像进行灰度化或 Gamma 归一化；
- 计算每个像素的水平和垂直梯度；
- 得到梯度幅值和梯度方向；
- 将图像划分成多个 Cell；
- 在每个 Cell 中建立梯度方向直方图；
- 将相邻 Cell 组成 Block；
- 对 Block 中的特征向量进行归一化；
拼接所有 Block 特征得到最终 HOG 描述子。

Cell 的作用

Cell 用于统计局部区域内的边缘方向分布。

Block 的作用

Block 对相邻 Cell 的特征进行局部归一化，从而减小光照、阴影和局部对比度变化的影响。

5. Faster R-CNN 基本流程

- 输入图像经过卷积网络，得到共享特征图；
- RPN 在特征图上生成并筛选候选区域；
- 将候选区域映射到共享特征图；
- ROI Pooling 将不同大小的候选区域转换为固定大小的特征；
- 全连接网络对候选区域进行分类和边界框回归；
使用非极大值抑制得到最终检测结果。

Anchor

Anchor 是在特征图每个位置预设的不同比例和尺寸的参考框。

RPN

RPN 负责：

- 判断 Anchor 是前景还是背景；
 - 预测 Anchor 到真实目标框的坐标偏移；
- 生成候选区域。

ROI Pooling

ROI Pooling 将不同大小的候选区域变成固定大小的特征，以便输入后续全连接层。

2.10 四、解答题参考答案

1. 二值形态学与 Hough 变换

(1) 膨胀结果

结构元素包含中心及其上、下、左、右位置。

每个前景像素都会向四个方向扩张一个像素，因此：

$$I \oplus B = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

腐蚀结果

只有当某个像素及其上、下、左、右位置全部为 1 时，该像素才会在腐蚀结果中保留。

因此：

$$I \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(2) Hough 变换

当 $\theta=0$

。

因为：

$$\cos 0^\circ = 1, \sin 0^\circ = 0$$

所以：

$$\rho = x$$

各点对应的 ρ 为：

$$1, 2, 3, 2, 2, 2$$

其中：

$$\rho = 2$$

获得 4 票，票数最多。

对应直线：

$$x = 2$$

$$\text{当 } \theta = 135$$

$$\cos 135^\circ = -\sqrt{2}/2, \sin 135^\circ = \sqrt{2}/2$$

因此：

$$\rho = (\sqrt{2}/2)(y - x)$$

对于点：

$$(1, 1), (2, 2), (3, 3)$$

均有：

$$\rho = 0$$

因此 $\rho = 0$ 获得 3 票。

对应直线：

$$0 = -x + y$$

即：

$$y = x$$

最终检测到的两条直线为：

$$x = 2 \quad y = x$$

2. CNN 尺寸和参数计算

(1) Conv1 输出尺寸

$$H_{\text{out}} = \text{floor}((64 + 2 \times 2 - 5)/2) + 1 = 32$$

因此输出特征图为：

$$32 \times 32 \times 8$$

(2) Conv1 参数量

每个卷积核连接 3 个输入通道：

$$5 \times 5 \times 3 = 75$$

加一个偏置，共 76 个参数。

一共有 **8** 个卷积核：

$$(5 \times 5 \times 3 + 1) \times 8 = 76 \times 8 = 608$$

(3) MaxPool 输出尺寸

池化窗口为 **2×2**，步长为 **2**：

$$16 \times 16 \times 8$$

(4) Conv2 输出尺寸

输入尺寸为 **16×16**，卷积核为 **3**，填充为 **1**，步长为 **1**：

$$H_{\text{out}} = (16 + 2 \times 1 - 3) / 1 + 1 = 16$$

因此：

$$16 \times 16 \times 16$$

(5) Conv2 参数量

每个卷积核参数量为：

$$3 \times 3 \times 8 + 1 = 73$$

共有 **16** 个卷积核：

$$73 \times 16 = 1168$$

(6) 全连接层参数量

Flatten 后的特征数量为：

$$16 \times 16 \times 16 = 4096$$

全连接层有 **10** 个输出：

$$4096 \times 10 + 10 = 40970$$

(7) 总参数量

$$608 + 1168 + 40970 = 42746$$

因此总参数量为：

$$42746$$

2.11 五、程序题参考答案

1. 手写 3×3 最大值滤波

```
#include <opencv2/opencv.hpp>
#include <algorithm>
#include <stdexcept>

cv::Mat maxFilter3x3(const cv::Mat& src)
{
    if (src.empty()) {
        throw std::invalid_argument(" 输入图像不能为空");
    }

    if (src.type() != CV_8UC1) {
        throw std::invalid_argument(
            " 输入图像必须是 8 位单通道灰度图像"
        );
    }

    // 边界默认复制原图像
    cv::Mat dst = src.clone();

    for (int row = 1; row < src.rows - 1; ++row) {
        for (int col = 1; col < src.cols - 1; ++col) {

            uchar maxValue = 0;

            // 遍历 3×3 邻域
            for (int dr = -1; dr <= 1; ++dr) {
                for (int dc = -1; dc <= 1; ++dc) {
                    uchar value =
                        src.at<uchar>(row + dr, col + dc);

                    maxValue = std::max(maxValue, value);
                }
            }

            dst.at<uchar>(row, col) = maxValue;
        }
    }

    return dst;
}
```

```
int main()
```

```

{

    cv::Mat src = cv::imread(
        "input.png",
        cv::IMREAD_GRAYSCALE
    );

    if (src.empty()) {
        return -1;
    }

    cv::Mat dst = maxFilter3x3(src);

    cv::imshow("Original", src);
    cv::imshow("Maximum Filter", dst);
    cv::waitKey(0);

    return 0;
}

```

核心逻辑

```

uchar maxValue = 0;

for (int dr = -1; dr <= 1; ++dr) {
    for (int dc = -1; dc <= 1; ++dc) {
        maxValue = std::max(
            maxValue,
            src.at<uchar>(row + dr, col + dc)
        );
    }
}

```

最大值滤波相当于灰度图像的膨胀运算，能够扩大图像中的亮区域。

2. 简化 U-Net 实现

```

import torch
import torch.nn as nn

class DoubleConv(nn.Module):
    """ 连续执行两次卷积、BN 和 ReLU。 """

    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()

```

```
        self.layers = nn.Sequential(
            nn.Conv2d(
                in_channels,
                out_channels,
                kernel_size=3,
                padding=1,
                bias=False
            ),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),

            nn.Conv2d(
                out_channels,
                out_channels,
                kernel_size=3,
                padding=1,
                bias=False
            ),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.layers(x)

class MiniUNet(nn.Module):
    def __init__(
        self,
        in_channels: int = 3,
        num_classes: int = 2
    ):
        super().__init__()

        # 编码器第一层
        self.encoder1 = DoubleConv(
            in_channels,
            64
        )

        self.pool = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        # 编码器第二层
        self.encoder2 = DoubleConv(
            64,
```

```
        128
    )

    # 上采样: 通道 128 变为 64
    self.up = nn.ConvTranspose2d(
        128,
        64,
        kernel_size=2,
        stride=2
    )

    # 拼接后通道数为 64 + 64 = 128
    self.decoder1 = DoubleConv(
        128,
        64
    )

    # 输出每个像素的类别分数
    self.output_conv = nn.Conv2d(
        64,
        num_classes,
        kernel_size=1
    )

def forward(self, x: torch.Tensor) -> torch.Tensor:
    # 编码器浅层特征
    feature1 = self.encoder1(x)

    # 下采样
    x = self.pool(feature1)

    # 深层特征
    x = self.encoder2(x)

    # 上采样
    x = self.up(x)

    # 跳跃连接: 沿通道维拼接
    x = torch.cat(
        [feature1, x],
        dim=1
    )

    # 解码器
    x = self.decoder1(x)

    # 输出分割结果
    x = self.output_conv(x)
```

```
        return x

if __name__ == "__main__":
    model = MiniUNet(
        in_channels=3,
        num_classes=4
    )

    image = torch.randn(
        2,
        3,
        256,
        256
    )

    output = model(image)

    print(" 输入尺寸: ", image.shape)
    print(" 输出尺寸: ", output.shape)
```

运行结果为：

输入尺寸: torch.Size([2, 3, 256, 256])

输出尺寸: torch.Size([2, 4, 256, 256])

输出中的 4 个通道表示每个像素属于 4 个不同类别的预测分数。U-Net 没有直接使用全连接层，因此能够保留图像的空间结构，实现逐像素分类。

第三编 深度学习专项题库

覆盖 CNN 组成与训练、AlexNet、VGG、GoogLeNet、ResNet、Faster R-CNN、Anchor、ROI Pooling 和 U-Net。

3.1 一、问答题专项

1. 简述卷积神经网络的基本组成及各部分作用

参考答案

卷积神经网络通常由以下部分组成：

卷积层：通过卷积核提取局部特征，例如边缘、纹理和高级语义特征。

激活函数：引入非线性，使网络能够学习复杂映射，常用 ReLU。

池化层：降低特征图空间尺寸，减少计算量，提高对局部位移的鲁棒性。

归一化层：稳定特征分布，加快网络训练，例如 Batch Normalization。

全连接层：将提取的特征映射为最终类别或回归结果。

输出层：根据任务输出类别概率、边界框或像素分类结果。

2. 卷积神经网络为什么采用局部连接和权值共享？

参考答案

局部连接是指一个输出神经元只与输入图像的局部区域连接，符合图像局部像素相关性较强的特点。

权值共享是指同一个卷积核在整幅图像上重复使用，使其能够在不同位置检测同一种特征。

二者的作用是：

- 大幅减少参数量；
- 降低计算复杂度；
- 能够检测不同位置的相同特征；

使卷积具有一定的平移等变性。

3. 卷积层与全连接层有什么区别？

参考答案

对比项 卷积层 全连接层

连接方式 局部连接 所有输入与所有输出连接

权值使用 权值共享 每条连接通常有独立权重

参数量 较少 通常较多

是否保留空间结构 保留 展平后通常不保留
主要作用 提取局部特征 完成分类或回归
卷积层适合处理图像等具有空间结构的数据；全连接层通常用于最终决策。

4. 如何计算卷积层输出尺寸和参数量？

参考答案

输入特征图空间尺寸为 H

$$H_{in} \times W_{in}$$

，卷积核大小为 K ，填充为 P ，步长为 S ，则：

$$H_{out} = \text{floor}((H_{in} + 2P - K)/S) + 1$$

宽度计算方法相同。

如果输入通道数为 C

in

，输出通道数为 C

out

，包含偏置时参数量为：

$$N = (K_h K_w C_{in} + 1) C_{out}$$

不包含偏置时为：

$$N = K_h K_w C_{in} C_{out}$$

卷积层参数量与输入图像的高、宽无关。

5. 池化层有什么作用？最大池化和平均池化有什么区别？

参考答案

池化层主要作用是：

- 减小特征图空间尺寸；
- 降低计算量和存储量；
- 扩大后续神经元的有效感受野；

提高对局部平移和形变的鲁棒性。

最大池化取邻域最大值，更强调显著特征。

平均池化取邻域平均值，更强调区域整体信息。

普通池化层一般没有可学习参数，也通常不改变通道数。

6. 为什么神经网络需要激活函数？

参考答案

如果网络中只有线性变换，那么多层线性变换仍可合并成一次线性变换，增加层数并不能提高网络表达复杂非线性关系的能力。

激活函数为网络引入非线性，使其能够学习复杂的分类边界和图像特征。

常见激活函数有：

- ReLU;
 - Sigmoid;
 - Tanh;
- Softmax。

7. ReLU 激活函数有什么特点？

参考答案

ReLU 定义为：

$$f(x)=\max(0,x)$$

优点：

- 计算简单；
- 正区间梯度为 1；
- 可以缓解梯度消失；
- 能产生稀疏激活；

通常比 **Sigmoid** 收敛更快。

缺点：

- 当输入长期为负时，输出和梯度均为 0；
- 可能出现“神经元死亡”问题。

8. 简述卷积神经网络的训练循环

参考答案

一个标准训练循环包括：

- 从数据集中读取一个批次的数据和标签；
- 将数据输入网络，完成前向传播；
- 根据预测结果和真实标签计算损失；
- 清空上一轮保留的梯度；
- 反向传播计算各参数梯度；
- 优化器根据梯度更新网络参数；
- 重复多个批次和多个 Epoch；

在验证集上评估模型的泛化性能。

PyTorch 中的核心顺序为：

```
optimizer.zero_grad()
outputs = model(inputs)
```

```
loss = criterion(outputs, labels)
```

```
loss.backward() optimizer.step()
```

9. 什么是过拟合？如何缓解？

参考答案

过拟合是指模型在训练集上表现很好，但在验证集或测试集上表现较差，说明模型记住了训练数据中的细节和噪声，泛化能力不足。

常用缓解方法：

- 增加训练数据；
- 数据增强；
- Dropout；
- 权重衰减；
- 早停；
- 降低模型复杂度；
- Batch Normalization；

使用预训练模型。

10. 简述 AlexNet 的主要特点

参考答案

AlexNet 的主要特点包括：

- 使用 ReLU 替代传统 Sigmoid 或 Tanh；
- 使用 Dropout 缓解全连接层过拟合；
- 使用数据增强；
- 使用最大池化；

采用较深的卷积神经网络完成大规模图像分类。

AlexNet 证明了深度卷积网络在大规模视觉任务上的有效性。

11. 简述 VGGNet 的主要特点

参考答案

VGGNet 的核心特点是大量堆叠 3×3 小卷积核。

多个小卷积核堆叠可以：

- 获得与大卷积核相近的感受野；
- 引入更多非线性激活；
- 减少部分参数；

使网络结构更加规则。

缺点是网络层数较多，全连接层参数量大，计算和存储成本较高。

12. 简述 GoogLeNet 中的 Inception 结构

参考答案

Inception 结构在同一层并行使用不同尺度的操作，例如：

- 1×1 卷积；
- 3×3 卷积；
- 5×5 卷积；

池化。

然后将各分支输出沿通道维拼接。

其作用是同时提取不同尺度的图像特征。

其中 1×1 卷积常用于：

- 调整通道数；
- 降低后续大卷积的计算量；

引入非线性。

13. 简述 ResNet 残差学习的基本思想

参考答案

普通网络直接学习目标映射 $H(x)$ ，ResNet 改为学习残差：

$$F(x) = H(x) - x$$

因此输出为：

$$y = F(x) + x$$

如果新增网络层没有学到有效映射，只需使：

$$F(x) \approx 0$$

残差块就近似恒等映射，不会明显破坏原有特征。

残差连接还为梯度提供了直接传播路径，可以缓解：

- 梯度消失；
- 深层网络退化；

深层网络难以优化的问题。

14. 什么是深层网络退化问题？它与过拟合是否相同？

参考答案

网络退化是指随着网络深度增加，训练误差反而升高，深层网络甚至不如浅层网络。

这并不一定是过拟合，因为退化问题中深层网络在训练集上的误差也可能更高。

ResNet 通过恒等映射和残差学习，使新增层至少能够接近恒等映射，从而缓解网络退化问题。

15. ResNet 中主分支和捷径分支尺寸不一致时怎么办？

参考答案

残差相加要求两条分支的形状完全相同。

如果通道数或空间尺寸不同，可以在捷径分支使用：

1×1

卷积，并设置适当的步长，实现：

- 通道数调整；
- 特征图下采样；

尺寸匹配。

之后再进行逐元素相加。

16. 图像分类、目标检测和图像分割有什么区别？**参考答案**

图像分类：

判断整幅图像属于哪个类别，只输出类别。

目标检测：

同时输出图像中目标的类别和边界框位置。

图像分割：

对每个像素进行分类，输出与输入图像空间对应的像素级结果。

因此三者输出信息从粗到细为：

整图类别 → 类别+位置 → 逐像素类别

17. 简述 R-CNN、Fast R-CNN 和 Faster R-CNN 的区别**参考答案**

R-CNN：

- 使用外部算法生成候选区域；
 - 每个候选区域分别经过 CNN；
- 存在大量重复卷积，速度慢。

Fast R-CNN：

- 整幅图像只进行一次卷积；
 - 候选区域映射到共享特征图；
 - 使用 ROI Pooling 获得固定尺寸特征；
- 仍依赖外部候选框算法。

Faster R-CNN：

- 使用 RPN 在共享特征图上生成候选区域；
 - 候选区域生成和检测共享卷积特征；
- 实现近似端到端训练。

18. 什么是 Anchor? 它有什么作用?

参考答案

Anchor 是在特征图每个位置预先设置的一组参考框，通常具有不同的：

- 尺寸；
- 长宽比。

对于每个 **Anchor**，**RPN** 预测：

- 它是前景还是背景；
- 它相对于真实框的坐标偏移。

Anchor 为不同大小、不同形状的目标提供边界框回归的初始参考。

19. 简述 RPN 的作用

参考答案

RPN 即区域建议网络，它在主干网络输出的共享特征图上滑动，对每个 **Anchor** 进行：

- 前景与背景分类；
- 边界框位置回归。

之后根据置信度和 **NMS** 筛选候选区域，提供给后续检测头。

因此 **RPN** 的核心输出包括：

- 目标性得分；
- 边界框偏移量；
- 候选区域 **Proposal**。

20. ROI Pooling 的作用和基本原理是什么?

参考答案

不同候选区域的空间大小不同，而全连接层需要固定长度的输入。

ROI Pooling 将每个候选区域划分为固定数量的子区域，例如 7×7 ，并在每个子区域内进行最大池化，最终得到固定大小的特征图。

作用是：

- 将不同大小的 **ROI** 转换为固定尺寸；
- 使其能够输入后续全连接分类和回归网络。

ROI Pooling 需要进行坐标取整，因此可能产生空间量化误差。

21. 简述 U-Net 的结构和工作过程

参考答案

U-Net 由编码器、解码器和跳跃连接组成。

编码器：

- 使用卷积提取特征；
- 使用池化降低空间尺寸；

逐渐获得高级语义信息。

解码器：

- 使用上采样或转置卷积恢复空间尺寸；

逐步生成像素级分类结果。

跳跃连接：

将编码器对应层的浅层特征与解码器特征拼接，融合定位细节和深层语义信息。

22. U-Net 为什么使用跳跃连接？

参考答案

编码器经过多次池化后：

- 语义信息增强；
- 空间分辨率降低；

目标边界和定位细节丢失。

编码器浅层特征保留了丰富的：

- 边缘；
- 纹理；

空间位置信息。

跳跃连接将浅层定位信息与解码器的高级语义特征融合，从而提高分割边界和小目标的恢复能力。

3.2 二、解答题专项

解答题 1 卷积输出尺寸和参数量

输入特征图为：

$$32 \times 32 \times 3$$

卷积层参数为：

- 卷积核大小： 5×5 ；
- 输出通道数：16；
- 步长：1；
- 填充：2；

包含偏置。

求输出特征图尺寸和参数量。

参考答案

输出高度：

$$H_{out} = (32 + 2 \times 2 - 5) / 1 + 1 = 32$$

输出宽度同样为 **32**，输出通道数为 **16**：

$32 \times 32 \times 16$

每个卷积核参数量为：

$5 \times 5 \times 3 + 1 = 76$

共有 **16** 个卷积核：

$76 \times 16 = 1216$

因此参数量为：

1216

解答题 2 多层 CNN 尺寸与参数

网络结构如下：

输入： $64 \times 64 \times 3$

Conv1:

3×3 卷积，输出通道 16

stride=1, padding=1

MaxPool:

2×2 , stride=2

Conv2:

5×5 卷积，输出通道 32

stride=1, padding=0

求各层输出尺寸以及两个卷积层的参数量，均包含偏置。

参考答案

Conv1

输出尺寸：

$$H_{out} = (64 + 2 - 3)/1 + 1 = 64$$

所以：

$64 \times 64 \times 16$

参数量：

$$(3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = 448$$

MaxPool

空间尺寸减半，通道数不变：

$32 \times 32 \times 16$

Conv2

输出空间尺寸：

$$H_{out} = (32 - 5)/1 + 1 = 28$$

所以：

$28 \times 28 \times 32$

参数量：

$$(5 \times 5 \times 16 + 1) \times 32 = 401 \times 32 = 12832$$

因此：

Conv2 参数量 = 12832

解答题 3 CNN 加全连接层

网络结构为：

输入： $28 \times 28 \times 1$

Conv:

5×5 卷积，输出通道 6

stride=1, padding=0

MaxPool:

2×2 , stride=2

Flatten

全连接层：

输出 10 个神经元

计算各层输出尺寸和总参数量。

参考答案

卷积层输出

$$28 - 5 + 1 = 24$$

因此：

$$24 \times 24 \times 6$$

卷积参数量：

$$(5 \times 5 \times 1 + 1) \times 6 = 26 \times 6 = 156$$

池化层输出

$$12 \times 12 \times 6$$

Flatten

$$12 \times 12 \times 6 = 864$$

全连接层参数量

$$864 \times 10 + 10 = 8650$$

总参数量

$$156 + 8650 = 8806$$

最终：

$$\text{总参数量} = 8806$$

解答题 4 判断卷积参数是否计算偏置

某卷积层输入通道为 64，输出通道为 128，卷积核大小为 3×3 。

分别计算：

- 包含偏置时的参数量；
- 不包含偏置时的参数量。

参考答案

卷积权重数量：

$$3 \times 3 \times 64 \times 128 = 73728$$

包含偏置时，每个输出通道有一个偏置：

$$73728 + 128 = 73856$$

因此：

无偏置

含偏置

$$\text{无偏置} = 73728; \text{含偏置} = 73856$$

解答题 5 残差块尺寸匹配

残差块输入为：

$$x \in \mathbb{R}^{(N \times 64 \times 56 \times 56)}$$

主分支第一层为：

3×3 卷积

输入通道 64

输出通道 128

$$\text{stride}=2 \text{ padding}=1$$

主分支最终保持 **128** 个通道。回答：

- 主分支输出尺寸；
- 原输入能否直接与主分支相加；
- 捷径分支应如何处理。

参考答案

主分支空间尺寸：

$$\text{floor}((56 + 2 - 3)/2) + 1 = 28$$

因此主分支输出：

$$N \times 128 \times 28 \times 28$$

原输入为：

$$N \times 64 \times 56 \times 56$$

通道数和空间尺寸均不同，不能直接相加。

捷径分支应使用：

- 1×1 卷积；
- 输出通道数 128；

步长 2。

得到：

$N \times 128 \times 28 \times 28$

之后再与主分支相加。

解答题 6 连续小卷积核的感受野

连续使用两个步长为 1 的 3×3 卷积，不使用池化。

求第二层输出神经元相对于原图的感受野大小。

参考答案

第一层 3×3 卷积的感受野为：

3×3

第二层每移动一个位置，对应第一层特征移动一个位置。第二层再使用 3×3 卷积，感受野每个方向增加 2：

$3 + 2 = 5$

因此两个连续的 3×3 卷积对应：

5×5

感受野。

这也是 VGG 使用多个小卷积核替代大卷积核的原因之一。

解答题 7 Anchor 数量计算

某特征图尺寸为：

40×30

在特征图的每个位置设置：

- 3 种尺度；
- 3 种长宽比。

求 Anchor 总数。

参考答案

每个位置 **Anchor** 数量：

$3 \times 3 = 9$

特征图位置总数：

$40 \times 30 = 1200$

因此 **Anchor** 总数：

$1200 \times 9 = 10800$

所以：
10800

解答题 8 IoU 计算

预测框左上角、右下角坐标为：

$$B_p = (1, 1, 5, 5)$$

真实框为：

$$B_g = (3, 2, 7, 6)$$

计算 IoU。

参考答案

预测框宽高：

$$5 - 1 = 4, 5 - 1 = 4$$

面积：

$$S_p = 16$$

真实框宽高：

$$7 - 3 = 4, 6 - 2 = 4$$

面积：

$$S_g = 16$$

交集区域左上角为：

$$(\max(1, 3), \max(1, 2)) = (3, 2)$$

右下角为：

$$(\min(5, 7), \min(5, 6)) = (5, 5)$$

交集宽高：

$$5 - 3 = 2, 5 - 2 = 3$$

交集面积：

$$S_{\text{intersection}} = 2 \times 3 = 6$$

并集面积：

$$S_{\text{union}} = 16 + 16 - 6 = 26$$

因此：

$$\text{IoU} = 6 / 26 = 3 / 13 \approx 0.231$$

所以：

$$\text{IoU} \approx 0.231$$

解答题 9 NMS 过程

同一类别有 4 个预测框：

框 置信度

A 0.95

B 0.88

C 0.80

D 0.60

已知：

$$\text{IoU}(A,B)=0.70 \quad \text{IoU}(A,C)=0.20 \quad \text{IoU}(A,D)=0.10 \quad \text{IoU}(C,D)=0.65$$

NMS 阈值为 0.5，求最终保留的框。

参考答案

首先选择置信度最高的 A 并保留。

B 与 A 的 IoU 为：

$$0.70 > 0.5$$

所以删除 B。

C 与 A 的 IoU 为：

$$0.20 < 0.5$$

保留候选资格。

D 与 A 的 IoU 为：

$$0.10 < 0.5$$

也暂时保留。

接下来选择剩余框中置信度最高的 C 并保留。

D 与 C 的 IoU 为：

$$0.65 > 0.5$$

因此删除 D。

最终保留：

A、C

解答题 10 ROI Pooling 计算

一个 **ROI** 在特征图上对应：

$$X = \begin{bmatrix} 1 & 5 & 7 & 2 \\ 4 & 6 & 3 & 5 \\ 2 & 8 & 9 & 1 \\ 3 & 2 & 4 & 6 \end{bmatrix}$$

将其 ROI Pooling 为 2×2 ，采用最大池化，求输出。

参考答案

将区域划分为 4 个 2×2 子区域。

左上区域：

$$\begin{bmatrix} 1 & 5 \\ 4 & 6 \end{bmatrix}$$

最大值为 6。

右上区域：

$$\begin{bmatrix} 7 & 2 \\ 3 & 5 \end{bmatrix}$$

最大值为 8。

左下区域：

$$\begin{bmatrix} 2 & 8 \\ 3 & 2 \end{bmatrix}$$

最大值为 7。

右下区域：

$$\begin{bmatrix} 9 & 1 \\ 4 & 6 \end{bmatrix}$$

最大值为 9。

所以输出为：

$$\begin{bmatrix} 6 & 7 \\ 8 & 9 \end{bmatrix}$$

解答题 11 U-Net 尺寸与通道计算

简化 **U-Net** 中：

输入： $3 \times 256 \times 256$

DoubleConv: $3 \rightarrow 64$

MaxPool: 2×2 , stride=2

DoubleConv: $64 \rightarrow 128$

转置卷积: $128 \rightarrow 64$ ，空间尺寸扩大 2 倍

与编码器第一层特征拼接

DoubleConv: $128 \rightarrow 64$

1×1 卷积: $64 \rightarrow 4$

求各主要阶段特征尺寸。

参考答案

输入:

$3 \times 256 \times 256$

第一层 **DoubleConv** 保持空间尺寸:

$64 \times 256 \times 256$

MaxPool 后:

$64 \times 128 \times 128$

第二层 **DoubleConv**:

$128 \times 128 \times 128$

转置卷积上采样并降通道:

$64 \times 256 \times 256$

与编码器第一层的:

$64 \times 256 \times 256$

沿通道维拼接:

$(64+64) \times 256 \times 256$

即:

$128 \times 256 \times 256$

解码器 **DoubleConv** 后:

$64 \times 256 \times 256$

最后 1×1 卷积输出:

$4 \times 256 \times 256$

表示每个像素对应 4 个类别分数。

解答题 12 训练代码顺序分析

下面训练代码存在错误:

```
for images, labels in train_loader:
    outputs = model(images)
    loss = criterion(outputs, labels)
    optimizer.step()
    loss.backward()
    optimizer.zero_grad()
```

指出错误并写出正确顺序。

参考答案

原代码的问题是:

- 在反向传播前调用了 `optimizer.step()`;
- 梯度还没有计算, 无法正确更新参数;

清空梯度的位置不合理。

正确顺序为：

```
for images, labels in train_loader:
    optimizer.zero_grad()

    outputs = model(images)
    loss = criterion(outputs, labels)

    loss.backward()
    optimizer.step()
```

对应过程：

梯度清零

→ 前向传播

→ 计算损失

→ 反向传播

→ 更新参数

3.3 三、综合性问答题

1. 为什么传统方法需要人工设计特征，而 CNN 可以自动学习特征？

参考答案

传统视觉方法通常需要人工设计边缘、纹理、角点、SIFT 或 HOG 等特征，再将特征输入分类器。

CNN 通过多层卷积从数据中自动学习特征：

- 浅层学习边缘和纹理；
- 中层学习局部形状和部件；

深度学习目标语义。

训练过程中，特征提取器和分类器根据同一损失函数共同优化，实现端到端学习。

2. 为什么图像分类网络不能直接完成精确图像分割？

参考答案

普通分类网络经过多次池化和下采样后，空间分辨率大幅降低，最终通常只输出整幅图像的类别。

图像分割需要恢复到原图大小，并对每个像素分类，因此需要：

- 保留空间信息；
- 使用上采样恢复尺寸；
- 融合浅层定位特征；

输出与输入空间对应的特征图。

U-Net 正是通过编码器、解码器和跳跃连接实现这一过程。

第四编 Faster R-CNN 专项题库

围绕 Anchor、RPN、Proposal、ROI Pooling 及其计算过程进行集中训练。

4.1 一、Anchor 专项问答题

1. 什么是 Anchor?

参考答案

Anchor 是在特征图的每一个空间位置预先设置的一组参考框。

这些参考框通常具有不同的：

- 尺寸；

长宽比。

RPN 以 Anchor 为基础，预测：

- 该 Anchor 属于前景还是背景；

该 Anchor 应如何平移和缩放，才能更接近真实目标框。

Anchor 不是最终检测框，而是边界框回归的初始参考。

2. 为什么需要设置多个尺度和长宽比的 Anchor?

参考答案

图像中的目标可能具有不同的大小和形状，例如：

- 行人通常较高、较窄；
- 汽车通常较宽；
- 远处目标较小；

近处目标较大。

在同一个特征图位置设置不同尺度和长宽比的 Anchor，可以使网络覆盖更多类型的目标框，提高候选框与真实目标的初始重合程度。

3. Anchor 是设置在原图上，还是设置在特征图上?

参考答案

Anchor 在计算上对应特征图的每个位置，但每个 Anchor 表示的是原始图像坐标系中的一个候选框。

例如，主干网络的总步长为 16，那么特征图上相邻两个位置，在原图中大约相距 16 个像素。

因此：

- Anchor 的中心由特征图位置决定；

- **Anchor** 的实际尺寸通常按照原图尺度定义；网络预测是在特征图上完成的。

4. **Anchor** 的数量如何计算？

参考答案

若特征图大小为：

$H \times W$

每个位置设置：

- S 种尺度；

R 种长宽比。

则每个位置的 **Anchor** 数为：

$k = S \times R$

总 **Anchor** 数为：

$H \times W \times k$

5. 什么是正 **Anchor**、负 **Anchor** 和忽略 **Anchor**？

参考答案

通常根据 **Anchor** 与真实框的 **IoU** 划分。

正 **Anchor**

满足以下情况之一：

- 与某个真实框的 **IoU** 高于正样本阈值；
- 是与某个真实框 **IoU** 最大的 **Anchor**。

正 **Anchor** 用于学习：

- 前景分类；

边界框回归。

负 **Anchor**

与所有真实框的 **IoU** 都较低，视为背景，只参与前景、背景分类。

忽略 **Anchor**

IoU 处于正负阈值之间，不参与当前损失计算，避免模糊样本干扰训练。

具体阈值可能因实现而不同。

6. 为什么与真实框 **IoU** 最大的 **Anchor** 通常要被设为正样本？

参考答案

某些真实目标可能没有任何 **Anchor** 达到规定的正样本 **IoU** 阈值。

如果只按照固定阈值划分，该目标可能完全没有正样本，网络就无法学习检测它。

因此，将与每个真实框 **IoU** 最大的 **Anchor** 设为正样本，可以保证每个真实目标至少对应一个正 **Anchor**。

7. RPN 对每个 Anchor 输出什么？

参考答案

RPN 对每个 Anchor 通常输出两类结果：

- 目标性分类结果：判断 Anchor 是前景还是背景；

边界框回归结果：预测 4 个坐标偏移量。

边界框回归通常包括：

t_x, t_y, t_w, t_h

分别控制：

- 中心横坐标移动；
- 中心纵坐标移动；
- 宽度缩放；

高度缩放。

8. 如果每个位置有 k 个 Anchor，RPN 分类分支和回归分支分别输出多少通道？

参考答案

若二分类采用每个 Anchor 两个分数，即前景和背景，则分类分支输出：

$2k$

个通道。

每个 Anchor 需要预测 4 个回归量，所以回归分支输出：

$4k$

个通道。

例如每个位置有 9 个 Anchor：

分类输出通道 $= 2 \times 9 = 18$

回归输出通道 $= 4 \times 9 = 36$

有些实现使用每个 Anchor 一个前景分数，此时分类输出为 k 个通道。

9. Anchor 与 Proposal 有什么区别？

参考答案

Anchor：

- 是预先设置的参考框；
- 数量很多；
- 位置和形状尚未根据图像内容调整；

是 RPN 预测的基础。

Proposal：

- 是 Anchor 经过边界框回归调整后的候选框；
- 通常还经过越界裁剪、低分框删除和 NMS 筛选；
- 更接近真实目标；

被送入 ROI Pooling 和后续检测头。

流程为：

Anchor

→ RPN 分类与回归

→ 调整后的候选框

→ NMS

→ Proposal

10. 为什么 RPN 之后还要进行 NMS？

参考答案

同一个目标附近通常会有许多相互重叠的 **Anchor**，它们经过回归后会产生大量相似的候选框。

NMS 保留得分最高的候选框，并删除与其高度重叠的其他框，从而：

- 减少重复 **Proposal**；
- 降低后续 **ROI** 处理的计算量；

保留更有代表性的候选区域。

11. Anchor 过多有什么优点和缺点？

参考答案

优点：

- 可以覆盖更多尺度和形状的目标；
- 提高目标与某个 **Anchor** 获得较高 **IoU** 的概率；

有利于检测尺度差异较大的目标。

缺点：

- 增加计算量和存储量；
- 产生大量背景负样本；
- 正负样本更加不平衡；

产生更多重复候选框。

因此 **Anchor** 数量需要在覆盖能力和计算成本之间平衡。

12. 为什么 Anchor 机制会造成正负样本不平衡？

参考答案

特征图上会产生大量 **Anchor**，但图像中的真实目标数量通常较少。

因此：

- 只有少数 **Anchor** 与真实框高度重合，成为正样本；

大部分 **Anchor** 都属于背景，成为负样本。

训练时通常不会使用全部 **Anchor**，而是按照一定比例采样正负样本，例如尽量保持一定的正负样本比例。

4.2 二、ROI Pooling 专项问答题

13. 什么是 ROI?

参考答案

ROI 是 Region of Interest, 即感兴趣区域。

在 Faster R-CNN 中, ROI 通常指 RPN 生成的 Proposal 映射到共享特征图后所对应的区域。

每个 **ROI** 可能具有不同的:

- 高度;
- 宽度;
- 位置。

14. 为什么需要 ROI Pooling?

参考答案

RPN 产生的候选区域大小不同, 但后续全连接层要求输入特征长度固定。

ROI Pooling 可以将任意大小的候选区域转换为固定大小的特征图, 例如:

7×7

因此不同尺寸的候选框都可以送入同一个分类和边界框回归网络。

15. 简述 ROI Pooling 的基本步骤。

参考答案

以输出 H

$$H_o \times W_o$$

为例:

- 将原图中的 Proposal 映射到共享特征图;
- 对 ROI 边界坐标进行量化取整;

将 ROI 划分为 H

$$H_o \times W_o$$

- 个子区域;
- 在每个子区域内进行最大池化;

得到固定大小的 H

$$H_o \times W_o$$

特征图。

若输入特征图有 C 个通道, 最终输出为:

$$C \times H_o \times W_o$$

16. ROI Pooling 是否把整个 ROI 缩放成固定大小?**参考答案**

从效果上看, 它把不同大小的 ROI 转换为固定大小特征, 但实现时不是简单地直接缩放图像。

ROI Pooling 通常是:

- 将 ROI 划分为固定数量的网格;
- 每个网格内进行最大池化;

每个网格产生一个输出值。

因此, 它本质上是分块最大池化。

17. ROI Pooling 为什么使用共享特征图?**参考答案**

如果每个候选框都单独通过 CNN, 会产生大量重复卷积计算, 这是 R-CNN 速度慢的主要原因之一。

Fast R-CNN 和 Faster R-CNN 采用:

- 整幅图像只经过一次主干网络;
- 所有 ROI 都从同一个共享特征图中提取特征。
这样可以显著减少重复计算, 提高检测速度。

18. ROI Pooling 的输出尺寸由什么决定?**参考答案**

ROI Pooling 的输出空间尺寸由预先指定的池化目标大小决定, 例如:

7×7

而不是由输入 ROI 本身的大小决定。

如果共享特征图通道数为 C , 则无论输入 ROI 大小如何, 输出始终为:

$C \times 7 \times 7$

19. ROI Pooling 会改变通道数吗?**参考答案**

通常不会。

ROI Pooling 只对每个通道中的空间区域分别进行池化, 因此:

- 输入有 C 个通道;

输出仍有 C 个通道。

它改变的是空间尺寸, 而不是通道数。

20. ROI Pooling 的主要缺点是什么？

参考答案

ROI Pooling 需要对：

- ROI 边界坐标；

子区域边界坐标

进行取整量化。

这种取整会导致 ROI 与原始目标之间出现空间偏移，产生量化误差。

对于普通目标检测问题影响可能较小，但对于需要精确像素定位的任务，例如实例分割，影响会更加明显。

21. ROI Pooling 和 ROI Align 有什么区别？

参考答案

ROI Pooling

- 对 ROI 边界取整；
- 对分块边界取整；
- 每个区域进行最大池化；

存在明显量化误差。

ROI Align

- 不进行粗糙取整；
- 在浮点坐标位置进行采样；
- 使用双线性插值计算特征值；

空间对齐更加准确。

ROI Align 常用于 Mask R-CNN 等对空间定位精度要求较高的网络。

22. ROI Pooling 与普通最大池化有什么区别？

参考答案

普通最大池化：

- 对整张特征图使用固定大小的滑动窗口；
- 各位置采用相同池化规则；

输出大小由窗口、步长和输入尺寸决定。

ROI Pooling：

- 每次只处理一个 ROI；
- ROI 大小可以不同；
- 根据 ROI 大小动态划分子区域；

所有 ROI 最终得到相同的输出尺寸。

23. ROI Pooling 与 SPP 有什么联系和区别？

参考答案

共同点：

- 都可以将不同大小的输入区域转换为固定长度特征；
- 都使用空间分块和池化。

区别：

SPP：

- 通常使用多个空间尺度；
- 例如 1×1 、 2×2 、 4×4 ；

将多尺度池化结果拼接。

ROI Pooling：

- 针对每个候选区域；
 - 通常池化为单一固定尺寸，如 7×7 ；
- 主要应用于 Fast R-CNN 和 Faster R-CNN。

4.3 三、综合问答题

24. 简述 Faster R-CNN 从输入图像到最终检测结果的完整流程。

参考答案

- 输入图像经过主干卷积网络，得到共享特征图；
 - RPN 在共享特征图的每个位置设置多个 Anchor；
 - RPN 对 Anchor 进行前景、背景分类和边界框回归；
 - 将调整后的候选框进行裁剪、筛选和 NMS，得到 Proposal；
 - 将 Proposal 映射到共享特征图；
 - 使用 ROI Pooling 将不同大小的 ROI 转换为固定大小特征；
 - 检测头对每个 ROI 进行目标类别分类；
 - 检测头进一步进行边界框回归；
- 使用置信度筛选和 NMS 得到最终检测结果。

25. Anchor、RPN、Proposal 和 ROI Pooling 之间是什么关系？

参考答案

它们之间的关系为：

特征图每个位置生成 Anchor

→ RPN 判断前景/背景并预测坐标偏移

→ Anchor 经过回归和 NMS 形成 Proposal

→ Proposal 映射到共享特征图成为 ROI

→ ROI Pooling 提取固定大小特征

→ 分类与边界框精修

Anchor 是初始参考框，RPN 负责筛选和调整 **Anchor**，**Proposal** 是 RPN 输出的候选框，**ROI Pooling** 负责提取固定尺寸的候选区域特征。

26. Faster R-CNN 为什么被称为两阶段检测器？

参考答案

第一阶段是 RPN：

- 判断哪些区域可能包含目标；
生成 **Proposal**。

第二阶段是检测头：

- 对每个 **Proposal** 进行具体类别分类；
对边界框进一步精修。

因此：

第一阶段：产生候选区域

第二阶段：分类和精确定位

所以 **Faster R-CNN** 属于两阶段目标检测器。

27. RPN 的“二分类”和最终检测头的“多分类”有什么区别？

参考答案

RPN 只关心 **Anchor** 中是否存在某种目标，因此进行：

前景与背景二分类

它不需要判断目标具体是人、车还是动物。

最终检测头需要判断 **Proposal** 属于哪个具体类别，因此进行：

背景 + 多个目标类别的多分类

例如检测 20 类目标时，检测头通常输出 21 类，包括 20 个目标类和 1 个背景类。

28. Faster R-CNN 为什么需要进行两次边界框回归？

参考答案

第一次回归由 RPN 完成：

- 从粗糙 **Anchor** 得到较好的 **Proposal**；
目标是找到可能存在物体的区域。

第二次回归由检测头完成：

- 在已获得的 **Proposal** 基础上进一步精修；
通常结合具体类别信息得到更准确的最终框。

因此两次回归分别负责：

粗定位 → 精确定位

4.4 四、计算与解答题

解答题 1 Anchor 数量计算

某共享特征图大小为：

50×40

每个位置设置 3 种尺度和 3 种长宽比。

求 Anchor 总数。

参考答案

每个位置的 **Anchor** 数量为：

$3 \times 3 = 9$

特征图位置数量为：

$50 \times 40 = 2000$

总 **Anchor** 数：

$2000 \times 9 = 18000$

因此：

18000

解答题 2 RPN 输出通道数

特征图每个位置设置 9 个 Anchor。

假设：

- 分类分支对每个 Anchor 输出前景、背景两个分数；

回归分支对每个 Anchor 输出 4 个偏移量。

求两个分支输出通道数。

参考答案

分类分支：

$2 \times 9 = 18$

回归分支：

$4 \times 9 = 36$

所以：

分类分支 18 通道，回归分支 36 通道

解答题 3 Anchor 回归计算

某 **Anchor** 为：

$(x_a, y_a, w_a, h_a) = (40, 50, 20, 10)$

RPN 预测：

$t_x = 0.2, t_y = -0.1, t_w = \ln 1.5, t_h = \ln 2$

采用：

$$x = x_a + t_x w_a, y = y_a + t_y h_a, w = w_a \exp(t_w), h = h_a \exp(t_h)$$

求调整后的候选框。

参考答案

中心横坐标：

$$x = 40 + 0.2 \times 20 = 44$$

中心纵坐标：

$$y = 50 - 0.1 \times 10 = 49$$

宽度：

$$w = 20 \exp(\ln 1.5) = 20 \times 1.5 = 30$$

高度：

$$h = 10 \exp(\ln 2) = 10 \times 2 = 20$$

所以：

$$(x, y, w, h) = (44, 49, 30, 20)$$

解答题 4 Anchor 正负样本判断

某 **Anchor** 与三个真实框的 **IoU** 分别为：

$$0.20, 0.72, 0.35$$

规定：

- 最大 IoU 不小于 0.7 为正样本；
 - 最大 IoU 小于 0.3 为负样本；
- 其余忽略。

判断该 **Anchor** 类别。

参考答案

该 **Anchor** 的最大 **IoU** 为：

$$\max(0.20, 0.72, 0.35) = 0.72$$

因为：

$$0.72 \geq 0.7$$

所以该 **Anchor** 为正样本。

正 **Anchor**

解答题 5 ROI Pooling 计算

某 **ROI** 对应的特征区域为：

$$X = \begin{bmatrix} 1 & 5 & 2 & 4 \\ 3 & 6 & 7 & 1 \\ 2 & 1 & 9 & 5 \\ 4 & 8 & 3 & 6 \end{bmatrix}$$

将其 ROI Pooling 为 2×2 ，采用最大池化。

参考答案

左上区域：

[[1, 5], [3, 6]]

最大值为 6。

右上区域：

[[2, 1], [4, 8]]

最大值为 8。

左下区域：

[[2, 4], [7, 1]]

最大值为 7。

右下区域：

[[9, 5], [3, 6]]

最大值为 9。

所以输出为：

[[6, 7], [8, 9]]

解答题 6 ROI Pooling 输出尺寸

共享特征图通道数为 256，对一个任意大小的 ROI 执行 7×7 ROI Pooling。

求输出特征图尺寸以及展平后的特征数量。

参考答案

ROI Pooling 不改变通道数，所以输出为：

$256 \times 7 \times 7$

展平后的特征数量为：

$256 \times 7 \times 7 = 12544$

所以：

12544

使用说明

本合集根据给定复习材料整理。练习版隐藏参考答案，含答案版保留完整答案与计算过程；两版题号和章节结构一致。